MATHEMATICS / INDEPENDENT STUDY

MM8036

Type Theory

Complete solutions

Original teaching material aligned with Stockholm University course descriptions.
Independently authored; not an official university publication.

Edition 1 · 24 October 2026

Contents

1 Unit 1 · Binding and substitution 3
2 Unit 2 · Reduction and confluence 6
3 Unit 3 · Simple types and judgments 9
4 Unit 4 · Normalization and constructive logic 12
5 Unit 5 · Dependent types 15
6 Unit 6 · Induction, identity, and universes 18
7 Unit 7 · Substitutions and semantics 21
8 Unit 8 · Categorical models and proof assistants 24
9 Mock assessment 1 27
10 Mock assessment 2 29

How to use this volume E problems practice definitions and standard arguments; C problems combine ideas; A problems require synthesis or generalization. These are study difficulty labels, not official SU grading criteria. Time estimates are guides, not deadlines. Write a complete attempt before opening hints. In the hints volume, read Hint 1 first and return to the problem before consulting Hint 2. Solutions give a decisive idea, a complete argument, and criteria for self-assessment.

1 Unit 1 · Binding and substitution

MM8036-U1.01 · E · 15 min
Learning objective: U1.1 Parsing

Fully parenthesize $\lambda x.xyz$ and $ab(cd)$.

Decisive idea. The abstraction body extends to the right.

The terms are $\lambda x.((xy)z)$ and $(ab)(cd)$. In the first, the entire application is the body; in the second, cd is one argument.

Check your argument. Show both application groupings and binder scope.

MM8036-U1.02 · E · 15 min
Learning objective: U1.1 Free variables

Compute $\mathrm{FV}(\lambda x.(xy)(\lambda y.yz))$.

Decisive idea. The inner binder removes only its own y.

The left application has free variables $\{x, y\}$; the inner abstraction has $\{z\}$. Their union is $\{x, y, z\}$; the outer binder removes x, giving $\{y, z\}$.

Check your argument. Do not remove the free y in the left subtree.

MM8036-U1.03 · E · 15 min
Learning objective: U1.1 Alpha equivalence

Which of $\lambda x.\lambda y.x$, $\lambda a.\lambda b.a$, and $\lambda a.\lambda b.b$ are alpha equivalent?

Decisive idea. Does the body refer to the outer or inner binder?

The first two are alpha equivalent by fresh renaming of both binders. The third refers to the inner binder, whereas the first two refer to the outer binder; capture-free renaming preserves that relation, so it is not equivalent.

Check your argument. Give the binding reason for the negative answer.

MM8036-U1.04 · E · 15 min
Learning objective: U1.2 Substitution

Compute $(\lambda y.xy)[x := yz]$.

Decisive idea. Rename it to fresh w.

First use $\lambda w.xw$. Substitution gives $\lambda w.(yz)w$. Both y and z remain free.

Check your argument. A fresh name and preserved free variables are essential.

MM8036-U1.05 · C · 30 min
Learning objective: U1.2 Substitution under shadowing

Compute $(\lambda x.(\lambda y.xy)z)[y := x]$ and explain why the replacement has no effect.

Decisive idea. Every y in this term is bound.

The free-variable set of the term is $\{z\}$. Substitution replaces only free occurrences, so the result is the original term up to alpha equivalence. A mechanical algorithm may rename binders while checking freshness, but that does not change the alpha class.

Check your argument. Do not confuse renaming during an algorithm with a semantic change.

MM8036-U1.06 · C · 30 min
Learning objective: U1.2 Exact free-variable formula

Prove: if $x \in \mathrm{FV}(t)$, then $\mathrm{FV}(t[x := u]) = (\mathrm{FV}(t) \setminus \{x\}) \cup \mathrm{FV}(u)$.

Decisive idea. Induct on syntax; in the application case at least one branch contains a free x.

Induct on t. For $t = x$ the formula is immediate. For an application, substitution and free variables distribute over the two branches; any branch without x is unchanged, while at least one branch contributes all of $\mathrm{FV}(u)$. Their union is the claimed set. For $t = \lambda y.s$, the hypothesis excludes $y = x$; rename y fresh for u and distinct from x. Apply induction to s and remove y. Freshness leaves $\mathrm{FV}(u)$ intact and gives precisely the stated formula. These exhaust the possible cases.

Check your argument. Explain why a substituted occurrence exists and why binder freshness matters.

MM8036-U1.07 · C · 30 min
Learning objective: U1.2 Closed substitutions

If t has free variables among $x_1, \ldots, x_n$ and each u_i is closed, prove simultaneous substitution of u_i for x_i closes t.

Decisive idea. Closed replacements cannot introduce new free variables.

Apply substitution sequentially in any fixed order. Each replacement removes its variable and adds no free variables. Because every replacement is closed, subsequent substitutions cannot alter an earlier replacement. After all n steps the free-variable set is empty; this also agrees with simultaneous substitution.

Check your argument. State why sequential and simultaneous replacement coincide here.

MM8036-U1.08 · C · 30 min
Learning objective: U1.2 Composition law

Verify the substitution-composition law for $t = xy$, $u = yz$, $v = w$, substituting first x then y.

Decisive idea. Evaluate both sides separately.

The left side is $((yz)y)[y := w] = (wz)w$. On the right, $t[y := w] = xw$ and $u[y := w] = wz$, so substitution for x again gives $(wz)w$. Freshness is satisfied for distinct x, y, z, w.

Check your argument. Record the conditions before using the law.

MM8036-U1.09 · C · 30 min
Learning objective: U1.1 Structural induction

Let size count one for each variable, application, and abstraction node. Prove $|t[x := u]| = |t| + k(|u| - 1)$, where k is the number of free occurrences of x in t.

Decisive idea. Renaming changes names but not node counts.

Induct on t. A variable x contributes $|u| = 1 + (|u| - 1)$; any other variable contributes one with $k = 0$. In an application, sizes add plus one and occurrence counts add, giving the formula from the two induction hypotheses. An abstraction binding x has $k = 0$ and is unchanged. Any other abstraction may first be renamed fresh; apply induction to its body and add one.

Check your argument. Include the binding-x case and the application node.

MM8036-U1.10 · A · 50 min
Learning objective: U1.2 Necessity of hypotheses

Construct a counterexample to the substitution-composition equation when $x \in \mathrm{FV}(v)$, even though $x \neq y$.

Decisive idea. Take $u = z$ and $v = x$ with three distinct names.

Choose distinct x, y, z and put $t = y, u = z, v = x$. The left side $(y[x := z])[y := x]$ is x. The right side $(y[y := x])[x := z[y := x]]$ is z. Distinct free variables are not alpha equivalent. Thus the freshness hypothesis on v cannot simply be deleted.

Check your argument. Evaluate both sides and distinguish alpha equivalence from renaming free variables.

MM8036-U1.11 · A · 50 min
Learning objective: U1.2 Commuting substitutions

Prove that substitutions for distinct x, y commute if $x \notin \mathrm{FV}(v)$ and $y \notin \mathrm{FV}(u)$: $(t[x := u])[y := v] = (t[y := v])[x := u]$ up to alpha equivalence. Explain why both side conditions are needed.

Decisive idea. Use the unchanged-term lemma on $u[y := v]$.

Composition gives $(t[y := v])[x := u[y := v]]$. Since y is not free in u, this is the right side. Without the first condition, take $t = y, u = z, v = x$: the results are x and z. Without the second, take $t = x, u = y, v = z$: the results are z and y. Choose distinct x, y, z in each example.

Check your argument. Prove the result and test the two conditions independently.

MM8036-U1.12 · A · 50 min
Learning objective: U1.1 Binding invariants

Use zero-based de Bruijn indices to encode the closed terms $\lambda x.\lambda y.x(yx)$ and $\lambda a.\lambda b.a(ba)$. Explain why this representation detects their alpha equivalence, and why a rule for free variables is still needed for open terms.

Decisive idea. The body is $1(01)$.

Both terms encode as two abstractions with body $1(01)$. Renaming a bound variable leaves the number of intervening binders unchanged, so alpha-equivalent terms have the same encoding. Conversely, for closed terms the index at each leaf identifies its binder, and choosing fresh names for the binder nodes reconstructs a representative unique up to alpha renaming. An open term has occurrences with no enclosing binder, so indices alone in this convention do not name its free variables; one needs a separate free-name convention or an external context.

Check your argument. Explain both directions for closed terms and the limitation for open terms.

2 Unit 2 · Reduction and confluence

MM8036-U2.01 · E · 15 min
Learning objective: U2.1 Beta reduction

Reduce $(\lambda x.x)(\lambda y.y)$ to normal form.

Decisive idea. Substitute the argument for x.

One step gives $\lambda y.y$. It has no application with an abstraction as function, so it is beta normal.

Check your argument. Justify normality as well as the reduction.

MM8036-U2.02 · E · 15 min
Learning objective: U2.1 Capture avoidance

Reduce $(\lambda x.\lambda y.x)y$.

Decisive idea. Rename the inner binder before contraction.

Rename the body to $\lambda z.x$; contraction then yields $\lambda z.y$. The free-variable set is $\{y\}$, whereas the incorrect answer $\lambda y.y$ is closed.

Check your argument. Preserve the free occurrence of the argument.

MM8036-U2.03 · E · 15 min
Learning objective: U2.1 Divergence

Let $\Delta = \lambda x.xx$ and $\Omega = \Delta\Delta$. Show an infinite beta sequence from Ω.

Decisive idea. The contractum is identical to the source.

$\Omega \to \Omega \to \Omega \to \ldots$, because substituting Δ into xx gives $\Delta\Delta$. Repeating this valid step gives an infinite sequence.

Check your argument. A self-loop is a reduction step even though the printed term repeats.

MM8036-U2.04 · E · 15 min
Learning objective: U2.1 Reduction strategies

Compare outermost reduction and call-by-value on $(\lambda x.\lambda z.z)\Omega$.

Decisive idea. Call-by-value evaluates the argument first.

Outermost reduction gives $\lambda z.z$ in one step. Call-by-value repeatedly reduces Ω and never applies the outer function. Thus existence of a normal form does not ensure termination of every strategy.

Check your argument. Name the strategy restriction causing divergence.

MM8036-U2.05 · C · 30 min
Learning objective: U2.2 Joining a fork

For $(\lambda x.xx)((\lambda y.y)z)$, contract either the outer redex or the argument redex first and exhibit a common reduct.

Decisive idea. Reduce both duplicated argument redexes.

The outer-first path gives $((\lambda y.y)z)((\lambda y.y)z)$, then $z((\lambda y.y)z)$, then zz. The argument-first path gives $(\lambda x.xx)z$, then zz. Both branches join even though their lengths differ.

Check your argument. List valid one-step moves on both paths.

MM8036-U2.06 · C · 30 min
Learning objective: U2.2 Normal-form uniqueness

Derive uniqueness of beta normal forms up to alpha equivalence from confluence.

Decisive idea. A normal term cannot take a positive number of beta steps.

If $t \overset{*}{\to} n$ and $t \overset{*}{\to} m$ with both normal, confluence yields r with $n \overset{*}{\to} r$ and $m \overset{*}{\to} r$. Both sequences must have length zero, so n, r, m are equal in the syntax modulo alpha equivalence.

Check your argument. Use normality at the final step, not termination of all terms.

MM8036-U2.07 · C · 30 min
Learning objective: U2.2 Conversion

Show that beta-convertible normal terms are alpha equivalent. You may use Church–Rosser for conversion.

Decisive idea. Church–Rosser replaces the zigzag by a common reduct.

If n and m are convertible, Church–Rosser gives r with $n \overset{*}{\to} r \overset{*}{\leftarrow} m$. Their normality forces both paths to be empty, hence their alpha classes agree. A direct appeal only to a common source would be insufficient until the conversion form of the theorem is established.

Check your argument. Distinguish a conversion zigzag from a reduction sequence.

MM8036-U2.08 · C · 30 min
Learning objective: U2.1 Weak versus strong normalization

Prove that $(\lambda x.\lambda z.z)\Omega$ is weakly normalizing but not strongly normalizing under full beta reduction.

Decisive idea. Full reduction may reduce inside the ignored argument.

A single outer contraction reaches normal form $\lambda z.z$, proving weak normalization. Repeatedly contract the occurrence of Ω in the argument without contracting the outside redex; this is an infinite full beta sequence. Hence the term is not strongly normalizing.

Check your argument. Provide witnesses for both assertions.

MM8036-U2.09 · C · 30 min
Learning objective: U2.2 Parallel reduction

Compute the complete development of $(\lambda x.xx)((\lambda y.y)z)$. Explain why it differs from contracting just one redex.

Decisive idea. The argument develops to z.

Using the complete-development clause for a redex, the developed body is xx and the developed argument is z, so the result is zz. A single outer beta step instead leaves two copies of the argument redex. Parallel development contracts the residual copies represented by the recursively developed argument.

Check your argument. Use the definition recursively rather than treating development as a strategy name.

MM8036-U2.10 · A · 50 min
Learning objective: U2.2 Confluence versus termination

Give a finite abstract reduction system that is confluent but has an infinite reduction sequence, and prove both claims.

Decisive idea. Use $a \to a$ and $a \to b$ with b normal.

Take objects $\{a, b\}$ and exactly the rules $a \to a$ and $a \to b$, with no step from b. Every reduct of b is b, while a reduct of a is either a or b. Any pair of reducts of a joins at b; this proves confluence. The sequence repeating $a \to a$ is infinite. Thus confluence alone does not imply strong normalization, even when every object has a normal form.

Check your argument. Check all possible source objects and reduct pairs.

MM8036-U2.11 · A · 50 min
Learning objective: U2.2 Substitution and reduction

Prove that if $u \to u'$, then $t[x := u] \xrightarrow{*} t[x := u']$. Explain why one step is generally insufficient.

Decisive idea. There is one copy of the step for each free occurrence of x.

For $t = x$ use the given step; for another variable use zero steps. In an application, apply the induction sequences in its left and then right subterm. Under a binder other than x, rename fresh for u, u' and lift the induction sequence under the abstraction. A binder x blocks substitution and needs zero steps. For $t = xx$, the substitution produces two copies of u, so reducing both may require two steps; if x is absent it requires none.

Check your argument. Include zero steps and duplicated occurrences.

MM8036-U2.12 · A · 50 min
Learning objective: U2.2 Distinguishing convertibility

Prove the closed terms $\lambda x.\lambda y.x$ and $\lambda x.\lambda y.y$ are not beta convertible. Why is their different behavior on some arguments not needed for this proof?

Decisive idea. Combine normal-form uniqueness for conversion with the de Bruijn invariant.

Both terms are beta normal. Their encodings have respective bodies 1 and 0, so they are not alpha equivalent. If they were beta convertible, Church–Rosser and normality would force alpha equivalence, a contradiction. Operational tests could provide intuition, but this syntactic invariant already proves nonconvertibility without choosing or interpreting arguments.

Check your argument. Establish normality and failure of alpha equivalence separately.

3 Unit 3 · Simple types and judgments

MM8036-U3.01 · E · 15 min
Learning objective: U3.1 Typing judgments

Give a derivation of $\lambda x.x : A \to A$ in the empty context.

Decisive idea. Discharge $x : A$ by abstraction.

The variable rule gives $x : A \vdash x : A$. The abstraction rule then gives $\vdash \lambda x.x : A \to A$. The premise explicitly contains the assumption discharged by the conclusion.

Check your argument. Show the premise context and discharged assumption.

MM8036-U3.02 · E · 15 min
Learning objective: U3.1 Application

In context $f : A \to B, x : A$, derive $fx : B$.

Decisive idea. Apply the arrow elimination rule.

The context contains $f : A \to B$ and $x : A$, giving both variable judgments. Arrow elimination yields $fx : B$ in the same context. No assumption is discharged.

Check your argument. The argument type must be exactly the arrow domain.

MM8036-U3.03 · E · 15 min
Learning objective: U3.1 Arrow association

Give a type and derivation for $\lambda x.\lambda y.x$.

Decisive idea. First abstract over y, then x.

From $x : A, y : B \vdash x : A$, abstract over y to get $x : A \vdash \lambda y.x : B \to A$, then over x to obtain $A \to (B \to A)$.

Check your argument. Do not confuse this with $(A \to B) \to A$.

MM8036-U3.04 · E · 15 min
Learning objective: U3.2 Substitution

Given $x : A \vdash x : A$ and $\vdash u : A$, calculate the conclusion of substitution.

Decisive idea. The result type remains A in simple typing.

Substitution gives $\vdash x[x := u] : A$, namely $\vdash u : A$. This smallest instance makes clear that the substituted assumption is removed from the context.

Check your argument. State both the result term and resulting context.

MM8036-U3.05 · C · 30 min
Learning objective: U3.1 Composition

Derive the type of $\lambda f.\lambda g.\lambda x.f(gx)$ from chosen types $f : B \to C, g : A \to B$.

Decisive idea. Discharge assumptions in reverse order.

In context $f : B \to C, g : A \to B, x : A$, obtain $gx : B$ and then $f(gx) : C$. Three abstractions yield $(B \to C) \to (A \to B) \to A \to C$, with right association throughout.

Check your argument. Show the two intermediate application judgments.

MM8036-U3.06 · C · 30 min
Learning objective: U3.2 Weakening

Prove admissibility of adding a fresh unused variable $y : B$ to a simply typed context.

Decisive idea. Rename abstraction binders to remain fresh for the new context.

Induct on the original typing derivation. For a variable, its original declaration is still present. The constant rule is context-independent. For application, weaken both premises and apply the same rule. For abstraction, rename its bound variable away from y, weaken the body derivation under the extended context, and reapply abstraction. The resulting term and type are unchanged.

Check your argument. Treat the binder case and freshness explicitly.

MM8036-U3.07 · C · 30 min
Learning objective: U3.2 Preservation

Prove preservation for a root redex $(\lambda x.t)u \to t[x := u]$ using inversion and substitution.

Decisive idea. The body premise has the exact assumption required by substitution.

If $\Gamma \vdash (\lambda x.t)u : B$, application inversion supplies some A with $\Gamma \vdash \lambda x.t : A \to B$ and $\Gamma \vdash u : A$. Abstraction inversion supplies $\Gamma, x : A \vdash t : B$. Substitution therefore gives $\Gamma \vdash t[x := u] : B$.

Check your argument. Identify the same domain type in both premises.

MM8036-U3.08 · C · 30 min
Learning objective: U3.2 Open terms and progress

Give a well-typed open term that is neither a call-by-value value nor able to step, under the lecture convention that values are abstractions and the inert base constant.

Decisive idea. Use $f : o \to o, x : o$.

In context $f : o \to o, x : o$, the term $fx : o$ is typable. It is not an abstraction or constant. Neither subterm steps, and f is not an abstraction, so no call-by-value rule applies. The closedness assumption in progress prevents this example.

Check your argument. Use the specified value convention and explain all possible step rules.

MM8036-U3.09 · C · 30 min
Learning objective: U3.1 Untypability of self-application

Prove $\lambda x.xx$ is untypable in finite simple types.

Decisive idea. Its type would have to equal an arrow having itself as domain.

Suppose the declaration of x is A. Typing xx requires a type B with $A = A \to B$: the function occurrence has type $A \to B$, while the argument occurrence has type A. A finite simple type has a finite number of constructor nodes, whereas $A \to B$ has strictly more nodes than A. Thus this equality is impossible.

Check your argument. Explain why one variable cannot receive unrelated types in the same context.

MM8036-U3.10 · A · 50 min
Learning objective: U3.2 Failure of subject expansion

Construct a beta step from an untypable term to a typable closed term. Prove both typing assertions.

Decisive idea. Use $(\lambda x.\lambda y.y)\Omega$.

Take $(\lambda x.\lambda y.y)\Omega$. It reduces to $\lambda y.y$, typable at $A \to A$. If the source were typable, application inversion would make $\Omega = \Delta\Delta$ typable and hence would make $\Delta = \lambda z.zz$ typable. The finite-type argument of Exercise U3.09 rules that out. Thus typing is preserved forward by reduction but need not be reflected backward.

Check your argument. Use inversion to justify untypability of the whole source.

MM8036-U3.11 · A · 50 min
Learning objective: U3.2 Substitution proof

Prove the application and abstraction cases of the substitution lemma, including why a binder may need renaming.

Decisive idea. For an abstraction, choose its binder fresh for the substituting term and distinct from the variable being removed.

Assume $\Gamma, x : A \vdash t : B$ and $\Gamma \vdash u : A$. For an application sr, inversion gives $s : C \to B$ and $r : C$ under the extended context; induction substitutes in both, and application yields $s[x := u]r[x := u] : B$. For $t = \lambda y.s$, rename y fresh. Its body has a derivation under $\Gamma, x : A, y : C$. Weaken the typing of u by $y : C$, apply induction to the body (using the harmless exchange of distinct simple declarations), then abstract over y. Capture avoidance ensures this abstraction is the intended substituted term.

Check your argument. Track contexts and justify weakening rather than assuming the argument is typable under any context.

MM8036-U3.12 · A · 50 min
Learning objective: U3.1 Typing restricts syntax

Prove that a closed beta-normal arrow-only term of base type o, in the language with exactly one inert constant $c : o$, must be c.

Decisive idea. A closed neutral spine must be headed by a constant.

A normal term is an abstraction or a variable/constant followed by zero or more arguments: otherwise an application with an abstraction head would be a redex. A term of base type cannot be an abstraction. Closedness excludes a variable at the head of the outer spine. The only constant is $c : o$, which cannot accept an argument because its type is not an arrow. Hence the spine has no arguments and the term is c. Bound variables inside possible arguments do not change the absence of a suitable outer head.

Check your argument. Use normality, closedness, and the constant's type separately.

4 Unit 4 · Normalization and constructive logic

MM8036-U4.01 · E · 15 min
Learning objective: U4.2 Curry–Howard

Write a proof term for $A \to B \to A$.

Decisive idea. Return the first assumption.

$\lambda a.\lambda b.a$ has the required type: under $a : A, b : B$ the variable a has type A, and two abstractions discharge b then a.

Check your argument. Explain assumption discharge.

MM8036-U4.02 · E · 15 min
Learning objective: U4.2 Conjunction

Construct $(A \times B) \to A$ and $(A \times B) \to B$.

Decisive idea. Use the two projections.

The functions are $\lambda p.\pi_1(p)$ and $\lambda p.\pi_2(p)$. On an input pair (a, b) they compute to a and b, respectively.

Check your argument. Give their computation on a constructor.

MM8036-U4.03 · E · 15 min
Learning objective: U4.2 Disjunction

Construct $A \to A + B$ and explain what data the output contains.

Decisive idea. Keep the supplied evidence for A.

Send a to the left injection of a. The output includes both the tag "left" and the evidence $a : A$; no evidence for B is required.

Check your argument. Do not replace a constructive sum by an untagged assertion.

MM8036-U4.04 · E · 15 min
Learning objective: U4.1 Normalization vocabulary

State weak and strong normalization and their implication relation.

Decisive idea. Every infinite path must be excluded for strong normalization.

Weak normalization means existence of a finite reduction to a normal form. Strong normalization means absence of every infinite reduction sequence. In the finitely branching term system, strong normalization implies weak normalization by repeatedly choosing a step until none is possible. The converse fails by the ignored-Ω example.

Check your argument. Give the direction and a counterexample to its converse.

MM8036-U4.05 · C · 30 min
Learning objective: U4.2 Negation
Construct $(A \to B) \to ((B \to 0) \to (A \to 0))$.
Decisive idea. Compose the two functions.
The term $\lambda f.\lambda n.\lambda a.n(fa)$ works. Under $f : A \to B, n : B \to 0, a : A$, application first gives $fa : B$ and then $n(fa) : 0$. Discharging the three assumptions yields the stated type.
Check your argument. This is constructive contraposition; no converse is asserted.

MM8036-U4.06 · C · 30 min
Learning objective: U4.2 Sum elimination
Construct $((A \to C) \times (B \to C)) \to ((A + B) \to C)$ and state its two computation cases.
Decisive idea. Apply the function matching the tag.
Given pair (f, g), send a left input a to fa and a right input b to gb. These clauses are the sum eliminator with constant result type C, and its beta rules give exactly those outputs.
Check your argument. Both branches must produce the same target type.

MM8036-U4.07 · C · 30 min
Learning objective: U4.1 Candidate closure
Explain why every variable belongs to every reducibility candidate defined in Lecture 7.
Decisive idea. A variable has no immediate reducts.
Variables are neutral. The condition that every immediate reduct be reducible holds vacuously because there are no such reducts. Candidate property (3) therefore puts any variable in $\mathcal{R}_A$ for every simple type A. This is a property of raw terms; it does not assign all types to a variable in a fixed typing context.
Check your argument. Distinguish reducibility membership from a typing judgment.

MM8036-U4.08 · C · 30 min
Learning objective: U4.1 Why strengthen induction
Show that strong normalization of s and t does not imply strong normalization of st.
Decisive idea. Take both terms to be $\Delta = \lambda x.xx$.
Set $s = t = \Delta = \lambda x.xx$. The term Δ has no beta redex, so each copy is strongly normalizing. Their application is Ω, which reduces to itself forever. Thus an application proof needs the arrow candidate condition controlling all reducible arguments, rather than only the termination of the function and argument separately.
Check your argument. Check normality of Δ and divergence of its application.

MM8036-U4.09 · C · 30 min
Learning objective: U4.2 Double negation

Construct $A \to ((A \to 0) \to 0)$ and explain why reversing the arrow requires a different argument.

Decisive idea. Return $\lambda n.na$.

The full term is $\lambda a.\lambda n.na$. It constructs double-negation evidence from $a : A$. For the reverse direction, an input of type $(A \to 0) \to 0$ expects a refuter of A; it does not directly supply a term of A. The construction just given cannot be run backwards, and the general reverse principle is not adopted as an axiom of the constructive core.

Check your argument. Give the term without claiming that failure to find a reverse term alone proves uninhabitability.

MM8036-U4.10 · A · 50 min
Learning objective: U4.1 Canonicity consequence

Using normalization, preservation, and U3.12, prove every closed term of type o reduces to c in the arrow-only calculus with sole constant $c : o$.

Decisive idea. Normalize first, preserve the type, then classify the normal form.

Strong normalization gives a finite reduction to some normal n. Repeated preservation gives $\vdash n : o$, and reduction preserves closedness by the substitution free-variable bound. U3.12 classifies such a closed normal term as c. Hence the original term reduces to c. Confluence additionally shows that any normal form reached is the same, but is not needed to establish existence of this reduction.

Check your argument. Make the closedness and type of the normal form explicit.

MM8036-U4.11 · A · 50 min
Learning objective: U4.2 Constructive equivalence

Construct maps in both directions between $(A + B) \to C$ and $(A \to C) \times (B \to C)$. Show the composites agree pointwise on constructor inputs.

Decisive idea. Test the resulting functions at left and right inputs.

Map h to the pair sending a to $h(\mathrm{inl}(a))$ and b to $h(\mathrm{inr}(b))$. Reverse a pair (f, g) by sum elimination. Starting with h, the composite applied to a left input computes to $h(\mathrm{inl}(a))$, and similarly for a right input. Starting with (f, g), its two resulting functions compute at any a, b to $f(a), g(b)$. This proves the asserted pointwise computation behavior; an identity of functions in intensional identity types would require additional reasoning and is not claimed here.

Check your argument. Separate pointwise computation from an assumed function-extensionality principle.

MM8036-U4.12 · A · 50 min
Learning objective: U4.1 Normalization proof dependencies

Reconstruct the abstraction case of the fundamental reducibility lemma. Identify exactly where the controlled beta-expansion lemma is used.

Decisive idea. Use two environment extensions: a fresh variable and an arbitrary reducible argument.

Given a reducible environment for the free context, rename the binder x fresh and let s be its substituted body. Extend the environment by mapping x to a fresh variable. The body induction hypothesis gives a strongly normalizing renamed copy of s, hence s and its abstraction are strongly normalizing. For any $u \in \mathcal{R}_A$, extend the environment instead by $x := u$; induction yields $s[x := u] \in \mathcal{R}_B$. Since s, u are strongly normalizing, controlled beta expansion gives $(\lambda x.s)u \in \mathcal{R}_B$. Both clauses of $\mathcal{R}_{A \to B}$ are now satisfied.

Check your argument. Supply both environment extensions and both parts of the arrow definition.

5 Unit 5 · Dependent types

MM8036-U5.01 · E · 15 min
Learning objective: U5.1 Pi formation

State the data needed to form $\Pi_{x:A} B(x)$ in context Γ.

Decisive idea. It may mention the new variable x.

One needs $\Gamma \vdash A$ type and $\Gamma, x : A \vdash B(x)$ type. With levels $A : U_i$ and $B(x) : U_j$, the product lies in $U_{\max(i,j)}$. The body family is not required to be constant.

Check your argument. State the extended context and level rule.

MM8036-U5.02 · E · 15 min
Learning objective: U5.1 Dependent application

If $f : \Pi_{x:A} B(x)$ and $a : A$, what is the type of fa?

Decisive idea. The result lies over a.

The type is $B(a)$, formally $B(x)[x := a]$. Using an unrelated fiber $B(b)$ would require additional equality evidence and transport.

Check your argument. Show the dependency in the result type.

MM8036-U5.03 · E · 15 min
Learning objective: U5.2 Sigma introduction

What must be supplied to construct a term of $\Sigma_{x:A} B(x)$?

Decisive idea. Supply $a : A$ and then $b : B(a)$.

The pair (a, b) inhabits the dependent sum when $a : A$ and $b : B(a)$. An arbitrary $b : B(c)$ cannot be paired with a unless the needed conversion or transport is provided.

Check your argument. State the exact fiber for the second component.

MM8036-U5.04 · E · 15 min
Learning objective: U5.2 Sigma projections

State the types and computation rules for the projections of $p : \Sigma_{x:A} B(x)$.

Decisive idea. Evaluate both projections on (a, b).

The first projection has type A, and the second has type $B(\pi_1(p))$. On a constructor pair, $\pi_1(a, b) \equiv a$ and $\pi_2(a, b) \equiv b$, with the second type converting to $B(a)$.

Check your argument. Include dependent typing, not just projection equations.

MM8036-U5.05 · C · 30 min
Learning objective: U5.1 Dependent identity

Derive the type of $\lambda A.\lambda x.x$ with $A : U_0$. Determine the level of its type.

Decisive idea. Use $U_0 : U_1$.

Under $A : U_0, x : A$, the variable rule gives $x : A$. Abstraction twice gives $\Pi_{A:U_0}(A \to A)$. The inner arrow is in U_0, but the outer Pi has domain U_0, which belongs to U_1; hence the complete type lies in U_1.

Check your argument. Do not infer that a quantification over all small types is itself small.

MM8036-U5.06 · C · 30 min
Learning objective: U5.2 Dependent pair map

Given $f : A \to A'$ and $g : \Pi_{x:A}(B(x) \to B'(fx))$, construct a map $\Sigma_{x:A}B(x) \to \Sigma_{y:A'}B'(y)$.

Decisive idea. Send (a, b) to (fa, gab).

On a pair (a, b), form first component $fa : A'$ and second component $gab : B'(fa)$. The Sigma introduction rule therefore gives the required output pair. Pair elimination extends this definition to every input.

Check your argument. Check the second component against the transformed first component.

MM8036-U5.07 · C · 30 min
Learning objective: U5.1 Dependent composition

Given $f : \Pi_{x:A}B(x)$ and $g : \Pi_{x:A}(B(x) \to C(x))$, derive a term of $\Pi_{x:A}C(x)$.

Decisive idea. Apply g at the same index as f.

The term is $\lambda x.gx(fx)$. In context $x : A$, dependent application gives $fx : B(x)$ and $gx : B(x) \to C(x)$, then ordinary application gives $C(x)$. Abstraction closes the derivation.

Check your argument. Keep both functions at the same index.

MM8036-U5.08 · C · 30 min
Learning objective: U5.2 Existential evidence

Explain the computational difference between $\Sigma_{n:\mathrm{Nat}}P(n)$ and $\Pi_{n:\mathrm{Nat}}P(n)$, and give their respective elimination uses.

Decisive idea. Project a Sigma; apply a Pi.

A Sigma term contains a particular number and evidence about that number. Its projections yield $n : \mathrm{Nat}$ and evidence of $P(n)$. A Pi term supplies evidence of $P(n)$ for each requested n and is eliminated by application to a chosen number. A single witness does not establish all instances.

Check your argument. Use the exact types of the extracted evidence.

MM8036-U5.09 · C · 30 min
Learning objective: U5.2 Constant families

Show that Sigma and Pi over constant families reproduce ordinary products and arrows.

Decisive idea. The substitution into a constant family changes nothing.

If $B(x) = B$ independently of x, a Sigma pair has components $a : A, b : B$, its projections have types A, B, and their beta rules are those of $A \times B$. A Pi term is an abstraction returning B, application returns B, and its beta rule is ordinary function substitution. Thus these are the dependent constructions specialized to constant families, subject to the chosen eta rules of the theory.

Check your argument. Do not silently add an eta equality absent from the chosen presentation.

MM8036-U5.10 · A · 50 min
Learning objective: U5.2 Constructive choice

Construct a function from $\Pi_{x:A}\Sigma_{y:B(x)}R(x,y)$ to $\Sigma_{f:\Pi_{x:A}B(x)}\Pi_{x:A}R(x,fx)$. Explain why this does not introduce classical choice.

Decisive idea. Define f by first projection and its evidence by second projection.

For input h, let $f = \lambda x.\pi_1(hx)$ and $r = \lambda x.\pi_2(hx)$. The second projection has type $R(x,\pi_1(hx))$, which converts to $R(x,fx)$ by the beta rule for f. Hence (f,r) has the target Sigma type. The premise is a function returning witness-and-proof pairs, not a mere truncated or classical assertion of existence; the construction reorganizes data already supplied.

Check your argument. Verify conversion of the dependent result and explain what the premise contains.

MM8036-U5.11 · A · 50 min
Learning objective: U5.2 Dependent currying

Construct maps between $\Pi_{p:\Sigma_{x:A}B(x)}C(\pi_1(p),\pi_2(p))$ and $\Pi_{x:A}\Pi_{y:B(x)}C(x,y)$. Verify their computation on pair inputs.

Decisive idea. For the reverse, eliminate the input pair.

Send h to $\lambda x.\lambda y.h(x,y)$. Conversely send k to the function defined on (x,y) by kxy. Starting from h, the round trip at (x,y) computes to $h(x,y)$; starting from k, the other round trip applied to x, y computes to kxy. These pointwise equations follow from Pi and Sigma beta rules. An internal equality of the functions is not being inferred without function extensionality.

Check your argument. Check the dependent codomain in each direction and limit the equality claim.

MM8036-U5.12 · A · 50 min
Learning objective: U5.1 Context dependency

Explain why exchanging $x : A, y : B(x)$ into $y : B(x), x : A$ is generally invalid, although weakening by a fresh declaration is often admissible. Give a concrete family example.

Decisive idea. Use $n : \mathrm{Nat}, v : \mathrm{Vec}(D,n)$.

For fixed $D : U_0$, consider $n : \mathrm{Nat}, v : \mathrm{Vec}(D,n)$. The declaration of v requires the previously available n to form its type. Placing $v : \mathrm{Vec}(D,n)$ first leaves n unbound, so the context is not well formed. A fresh independent declaration can instead be inserted where all declarations remain well formed, and terms may be weakened accordingly. Exchange is admissible only when the dependency conditions permit it, unlike arbitrary exchange in a simple-type context.

Check your argument. Identify a formation failure, not merely an unconventional order.

6 Unit 6 · Induction, identity, and universes

MM8036-U6.01 · E · 15 min
Learning objective: U6.1 Addition computation

With addition recursive in its first argument, compute $\mathrm{add}(\mathrm{suc}(\mathrm{zero}), n)$.
Decisive idea. Keep the variable n untouched.
It reduces to $\mathrm{suc}(\mathrm{add}(\mathrm{zero}, n))$, then to $\mathrm{suc}(n)$. No inspection of n is necessary.
Check your argument. Show both computation steps.

MM8036-U6.02 · E · 15 min
Learning objective: U6.1 Induction motive

State the motive for proving right-zero addition.
Decisive idea. Use an identity type indexed by n.
The motive is $P(n) = \mathrm{Id}_{\mathrm{Nat}(\mathrm{add}(n,\mathrm{zero}),n)}$. At zero it reduces to reflexive equality; at successor the induction hypothesis can be lifted by congruence of successor.
Check your argument. State a family of types rather than a single fixed goal.

MM8036-U6.03 · E · 15 min
Learning objective: U6.2 Transport typing

For $p : \mathrm{Id}_{A(a,b)}$ and $u : B(a)$, state the type of $\mathrm{transport}_{B(p,u)}$ and its reflexivity computation.
Decisive idea. At reflexivity the endpoints coincide.
The result has type $B(b)$. When p is refl_a, transport computes to $u : B(a)$.
Check your argument. Specify direction from a to b.

MM8036-U6.04 · E · 15 min
Learning objective: U6.2 Universe levels

If $A : U_1$ and $B(x) : U_0$, find a universe containing $\Pi_{x:A} B(x)$ and one containing $\mathrm{Id}_{A(a,b)}$.
Decisive idea. Both depend on the level of A.
Both types belong to U_1: the Pi rule gives $\max(1, 0) = 1$, and identity on a type in U_1 stays in U_1. No cumulativity is needed for these formation judgments.
Check your argument. Use the stated noncumulative conventions.

MM8036-U6.05 · C · 30 min
Learning objective: U6.1 Associativity of addition

Prove addition associative by induction on its first argument.

Decisive idea. Lift the induction equality through successor.

For fixed n, k, induct on m with motive $\mathrm{Id}(\mathrm{add}(\mathrm{add}(m,n),k), \mathrm{add}(m,\mathrm{add}(n,k)))$. At zero both sides compute to $\mathrm{add}(n,k)$, so reflexivity suffices. At successor both sides compute to the successor of the respective sides for m. Apply congruence of successor to the induction hypothesis.

Check your argument. Reduce both sides before selecting the equality proof.

MM8036-U6.06 · C · 30 min
Learning objective: U6.1 Vector append

Describe and type both clauses of append on length-indexed vectors.

Decisive idea. The nil case length converts from $0 + n$ to n.

Append has type $\mathrm{Vec}(A,m) \to \mathrm{Vec}(A,n) \to \mathrm{Vec}(A,\mathrm{add}(m,n))$. At nil, return the second vector. At $\mathrm{cons}(a,v)$, return $\mathrm{cons}(a,\mathrm{append}(v,w))$. The recursive result has length $\mathrm{add}(m,n)$ for the tail length m, so the constructor has length $\mathrm{suc}(\mathrm{add}(m,n))$, judgmentally equal to $\mathrm{add}(\mathrm{suc}(m),n)$.

Check your argument. Check index conversion in both clauses.

MM8036-U6.07 · C · 30 min
Learning objective: U6.2 Symmetry by J

Derive symmetry of identity with an explicit motive and reflexive case.

Decisive idea. Set $C(a,b,p) = \mathrm{Id}_{A(b,a)}$.

Use the motive $C(a,b,p) = \mathrm{Id}_{A(b,a)}$. Path induction with this motive reduces the construction to $C(a,a,\mathrm{refl}_a) = \mathrm{Id}_{A(a,a)}$. Supply refl_a. The eliminator then defines symmetry for every $p : \mathrm{Id}_{A(a,b)}$ and computes to reflexivity on a reflexive input.

Check your argument. Allow the endpoint b to vary in the motive.

MM8036-U6.08 · C · 30 min
Learning objective: U6.2 Congruence

For $f : A \to D$, derive $\mathrm{Id}_{A(a,b)} \to \mathrm{Id}_{D(f(a),f(b))}$. Why is this an implication about identity proofs rather than an equality-reflection rule?

Decisive idea. The reflexive case is reflexivity at $f(a)$.

Take motive $C(a,b,p) = \mathrm{Id}_{D(f(a),f(b))}$ and reflexive data $\mathrm{refl}_{f(a)}$. J supplies the function. Its result is a term of an identity type; it does not add the conversion judgment $f(a) \equiv f(b)$ for an arbitrary variable p.

Check your argument. Distinguish an internal proof from a judgmental conversion.

MM8036-U6.09 · C · 30 min
Learning objective: U6.1 Safe head

Explain why a head function $\mathrm{Vec}(A, \mathrm{suc}(n)) \to A$ needs no nil branch. Can the same total signature be given for all lengths without extra data?

Decisive idea. Nil has length zero.

Only a cons constructor can inhabit a successor-length vector in this inductive family, so head returns its first component. A function on all lengths would also have to handle nil; with arbitrary A there may be no element to return, so such a uniform total head needs a nonempty-length condition, a default element, or an optional result type.

Check your argument. Use the index, not an unchecked runtime assumption.

MM8036-U6.10 · A · 50 min
Learning objective: U6.2 Transport composition

For $p : \mathrm{Id}_{A(a,b)}$ and $q : \mathrm{Id}_{A(b,c)}$, prove transport along their composite agrees propositionally with successive transport on $u : B(a)$.

Decisive idea. After both become reflexivity, both expressions compute to u.

Use path induction on p and then q, with the target family asserting equality in $B(c)$ between $\mathrm{transport}_{B(\mathrm{trans}(p,q),u)}$ and $\mathrm{transport}_{B\left(q,\mathrm{transport}_{B(p,u)}\right)}$. Generalize the varying endpoints and retain u in its proper starting fiber. At $p = q = \mathrm{refl}$, transitivity and both transports reduce, leaving $\mathrm{Id}_{B(a)}(u, u)$, inhabited by reflexivity. J lifts this base case to all endpoints and paths.

Check your argument. State the common result fiber and do not assume general paths compute.

MM8036-U6.11 · A · 50 min
Learning objective: U6.2 Judgmental and propositional equality

Compare the left-zero and right-zero laws for addition. Explain why a proof of right-zero does not make its open expression reduce by definition.

Decisive idea. A variable in the first argument blocks computation.

Left-zero $\mathrm{add}(\mathrm{zero}, n) \equiv n$ follows directly by the defining equation. Right-zero for variable n requires induction and produces a term of $\mathrm{Id}(\mathrm{add}(n, \mathrm{zero}), n)$. The recursive definition still cannot select a constructor of a variable, so the expression remains stuck under judgmental reduction. Equality elimination may transport along the proof, but equality reflection is not a rule of this theory.

Check your argument. Keep reduction, propositional proof, and transport distinct.

MM8036-U6.12 · A · 50 min
Learning objective: U6.2 Diagnosing misuse of path induction

A student argues: "Apply J to $p : \mathrm{Id}_{A(a,a)}$, so every such p equals reflexivity." Identify the missing motive issue; explain what J actually supplies.

Decisive idea. The alleged comparison to reflexivity is not well typed when the endpoint varies.

In based path induction one must define a motive for arbitrary $b : A$ and $p : \mathrm{Id}_{A(a,b)}$. The proposed equality between p and refl_a is not an identity type in one common fiber for arbitrary b, because reflexivity inhabits $\mathrm{Id}_{A(a,a)}$ rather than $\mathrm{Id}_{A(a,b)}$. Restricting the motive to $b = a$ before invoking J changes the eliminator. J permits valid families over all such endpoints, yielding transport and congruence; it does not by this argument yield the stronger K/UIP principle, which we have not assumed.

Check your argument. Identify the exact ill-typed comparison rather than merely naming UIP.

7 Unit 7 · Substitutions and semantics

MM8036-U7.01 · E · 15 min
Learning objective: U7.1 Context substitution

Describe a substitution from Δ to the context $x : A, y : B(x)$.

Decisive idea. The second term is checked after the first substitution.

It consists of $a : A$ in Δ and $b : B(a)$ in Δ. The second component cannot be checked in the uninstantiated family without accounting for $x := a$.

Check your argument. State both component types in the source context.

MM8036-U7.02 · E · 15 min
Learning objective: U7.1 Identity substitution

Write the identity substitution on $x : A, y : B(x)$.

Decisive idea. Check the dependent second component.

It is the tuple (x, y). The first component has type A and the second type $B(x)$, so both match their declarations; applying it leaves terms and types unchanged.

Check your argument. Check the dependency as well as the names.

MM8036-U7.03 · E · 15 min
Learning objective: U7.1 Composition order

For $\sigma : \Delta \to \Gamma$ and $\tau : \Theta \to \Delta$, state the source and target of $\sigma \circ \tau$ and the substitution equation for a term t over Γ.

Decisive idea. First reinterpret t over Δ, then over Θ.

The composite is $\Theta \to \Gamma$ and $t[\sigma][\tau] = t[\sigma \circ \tau]$. The rightmost substitution acts on the components supplied by the earlier one.

Check your argument. Do not reverse the order of context maps.

MM8036-U7.04 · E · 15 min
Learning objective: U7.2 Meaning explanations

Contrast a canonical natural-number term with a neutral natural-number term.

Decisive idea. A variable need not compute to a visible numeral in an open context.

Zero and successor constructor forms are canonical introductions for naturals. A variable $n : \mathrm{Nat}$ is neutral: no reduction selects whether it is zero or successor. In open reasoning it still has a valid type; "not a visible constructor" does not mean "ill typed."

Check your argument. Separate canonical shape from typing validity.

MM8036-U7.05 · C · 30 min
Learning objective: U7.1 Associativity on components

Prove associativity of context-substitution composition from the term substitution law.

Decisive idea. Both composites substitute twice into the same supplied term.

Let $\sigma : \Delta \to \Gamma, \tau : \Theta \to \Delta, \rho : \Xi \to \Theta$. If the component of σ at a declaration of Γ is s, the component of $(\sigma \circ \tau) \circ \rho$ is $s[\tau][\rho]$. The component of $\sigma \circ (\tau \circ \rho)$ is $s[\tau \circ \rho]$. The term substitution law equates these, with dependent types transported by the same law at each declaration. Thus the tuples agree componentwise.

Check your argument. Check component types, not just the first component's syntax.

MM8036-U7.06 · C · 30 min
Learning objective: U7.1 Projection and pairing

For context extension $\Gamma \cdot A$, verify $p \circ \langle \sigma, t \rangle = \sigma$ and $q[\langle \sigma, t \rangle] = t$ using tuple substitutions.

Decisive idea. Projection discards that last component; the generic variable selects it.

The tuple $\langle \sigma, t \rangle$ lists the components of σ followed by t. Applying projection removes the final entry and leaves σ. Substituting the tuple into the new last variable q selects its last entry t. The type of this last entry is $A[\sigma]$, as required.

Check your argument. Include the dependent type of the selected entry.

MM8036-U7.07 · C · 30 min
Learning objective: U7.2 Families of sets

In the set interpretation, describe the meaning of a dependent sum for $A = \{0, 1\}$, $B(0) = \{a\}$, $B(1) = \{b, c\}$. List its elements.

Decisive idea. It is a disjoint union of fibers.

The sum is $\{(0, a), (1, b), (1, c)\}$. The first projection records the index; the second projection is an element of the fiber determined by that first projection. Forgetting tags can conflate elements from different fibers in general.

Check your argument. List tagged pairs and connect them to dependent projection types.

MM8036-U7.08 · C · 30 min
Learning objective: U7.2 Dependent products of sets

For the same two fibers as U7.07, list all dependent functions and count them.

Decisive idea. The value at zero is forced; at one there are two choices.

There are two functions: one with values $f(0) = a, f(1) = b$, the other with $g(0) = a, g(1) = c$. Each is a section assigning an element of the appropriate fiber. The count is $1 \cdot 2 = 2$, not the sum of fiber sizes.

Check your argument. A dependent function makes a simultaneous choice in every fiber.

MM8036-U7.09 · C · 30 min
Learning objective: U7.2 Semantic substitution

Explain why interpreting $t[\sigma]$ agrees with evaluating the interpretation of t in the environment determined by σ. Prove the variable and application cases.

Decisive idea. For a variable, the tuple component is selected on either side.

The variable case reads the corresponding component of the substitution environment, exactly the interpretation of that component term. For application su, the induction hypotheses identify the interpretations of both substituted subterms with evaluation in the new environment; applying the interpreted function to the interpreted argument gives equality of the application interpretations. The abstraction case additionally extends the environment with the bound argument, which is why capture avoidance and lifted substitution matter.

Check your argument. Show the two required cases and identify the binder issue without omitting it from the general claim.

MM8036-U7.10 · A · 50 min
Learning objective: U7.2 Soundness is not completeness

The set model interprets every identity proof between fixed endpoints, when one exists, by the same element. Explain why soundness alone does not prove syntactic UIP.

Decisive idea. Its implication points in only one direction.

Soundness says a syntactic derivation has a valid interpretation. The interpretation may identify syntactically distinct proofs, because a model is allowed to forget distinctions. Deducing a syntactic UIP derivation from semantic identification would require an additional completeness or reflection theorem strong enough for that judgment. No such theorem has been established here, so the model's proof irrelevance cannot be imported as a rule of the paper theory.

Check your argument. State the direction of soundness and the missing converse.

MM8036-U7.11 · A · 50 min
Learning objective: U7.1 Explicit substitution design

Why does the equation $(t[\sigma])[\tau] = t[\sigma \circ \tau]$ not by itself prove that an explicit-substitution rewrite system terminates? Construct a simple warning using orientations.

Decisive idea. Allow both directions and a cycle appears.

If both directions are permitted as reductions, the two expressions alternate forever whenever they are syntactically distinct. Even choosing one orientation for this equation does not settle interactions with all other rules. Termination requires a well-founded measure or another proof covering the actual oriented rule set. The typed equations in the lecture specify substitution behavior, not a termination theorem for every implementation of it.

Check your argument. Distinguish equational specification from an oriented operational system.

MM8036-U7.12 · A · 50 min
Learning objective: U7.2 Reindexing a family

Let $A = \{0, 1\}$ with fibers as in U7.07, and let $\sigma : \{r, s, t\} \to A$ send r, s to zero and t to one. Describe the reindexed family, its total space, and its number of sections.

Decisive idea. Two singleton fibers and one two-element fiber remain.

The new fibers are $B(\sigma(r)) = \{a\}$, $B(\sigma(s)) = \{a\}$, and $B(\sigma(t)) = \{b, c\}$. The total space consists of $(r, a), (s, a), (t, b), (t, c)$, four tagged elements. A section must choose a at both r, s and choose either b or c at t, so there are two sections. This makes semantic substitution concrete: the index map changes where the fibers are read.

Check your argument. Retain separate tags even when two fibers happen to be the same set.

8 Unit 8 · Categorical models and proof assistants

MM8036-U8.01 · E · 15 min
Learning objective: U8.1 Context morphisms

Explain how contexts and substitutions form a category, naming identities and composition.

Decisive idea. Objects are well-formed contexts; arrows supply their variables.

A morphism $\Delta \to \Gamma$ is a well-typed simultaneous substitution for the declarations of Γ by terms over Δ, modulo the chosen definitional equality. Identity lists the context variables; composition substitutes one tuple into another. The identity and associativity equations establish the category laws.

Check your argument. Include the equality convention for arrows.

MM8036-U8.02 · E · 15 min
Learning objective: U8.1 Terminal context

Why is the empty context terminal in the substitution category?

Decisive idea. There is exactly one empty tuple from any context.

A substitution from Γ to the empty context supplies no components and therefore is unique. This is the terminal-object property; the direction is from each context into the empty one.

Check your argument. Do not reverse the direction and claim initiality.

MM8036-U8.03 · E · 15 min
Learning objective: U8.1 Evaluation

In sets, write evaluation $B^A \times A \to B$ and the curry of $h : X \times A \to B$.

Decisive idea. Fix the first coordinate of h.

Evaluation sends (f, a) to $f(a)$. The curry sends x to the function $a \mapsto h(x, a)$. Consequently evaluation of the curry at (x, a) returns $h(x, a)$, the beta equation.

Check your argument. Give the domains as well as the formulas.

MM8036-U8.04 · E · 15 min
Learning objective: U8.2 Checker contract

What does successful checking of a lab's theorem establish, and what does it leave to the author?

Decisive idea. The checker validates the term against the actual type under its rules.

It establishes that the supplied term is accepted at the written type with the checker options and definitions in use. The author must still ensure that the type accurately expresses the intended mathematical statement and that the chosen foundational rules match the intended setting. A checked trivial or misstated proposition is still a miscommunication.

Check your argument. Avoid both dismissing checking and overstating what it guarantees.

MM8036-U8.05 · C · 30 min
Learning objective: U8.1 Exponential uniqueness

In sets, prove the curry in U8.03 is the unique $k : X \to B^A$ satisfying $\text{eval} \circ (k \times 1_A) = h$.

Decisive idea. The equation forces $k(x)(a) = h(x, a)$.

For each x, the condition fixes the value of the function $k(x)$ at every a to be $h(x, a)$. Equality of set functions is pointwise equality, so $k(x)$ equals the stated curry at each x, and hence k is unique. This is semantic equality of functions in sets, not an added syntactic function-extensionality axiom.

Check your argument. Prove uniqueness in the indicated semantic category.

MM8036-U8.06 · C · 30 min
Learning objective: U8.1 Beta soundness

Explain categorically why interpreting an application of an abstraction satisfies the beta equation.

Decisive idea. Use the evaluation–curry identity and reindex along the argument.

If the body denotes $h : \Gamma \times A \to B$, abstraction denotes its curry $\Gamma \to B^A$. Pair this with an argument $u : \Gamma \to A$ and evaluate. The evaluation–curry law says the result is $h \circ \langle 1_\Gamma, u \rangle$, precisely the body's interpretation after substituting u for its added variable. Thus the semantic beta equation follows from the exponential universal property and semantic substitution.

Check your argument. Name the map representing substitution into the body.

MM8036-U8.07 · C · 30 min
Learning objective: U8.1 Comprehension in sets

For a set Γ and family $A(\gamma)$, construct the context extension and prove its pairing property.

Decisive idea. A point consists of an environment and a value in its fiber.

Set $\Gamma \cdot A = \{(\gamma, a) : \gamma \in \Gamma, a \in A(\gamma)\}$. Projection sends the pair to γ, and the generic term returns a in that fiber. Given $\sigma : \Delta \to \Gamma$ and $t(\delta) \in A(\sigma(\delta))$, the unique pairing sends δ to $(\sigma(\delta), t(\delta))$. Both projection equations hold by evaluation; any map satisfying them has these same two coordinates and is therefore equal.

Check your argument. Check fiber membership and uniqueness.

MM8036-U8.08 · C · 30 min
Learning objective: U8.2 Debugging an induction

An attempted proof of right-zero addition uses reflexivity for the variable case. Explain the failure and repair it using the lab definitions.

Decisive idea. Split into zero and successor and use congruence.

For variable n, $n + 0$ does not reduce to n, so reflexivity does not solve the target by conversion. Pattern match on n. At zero use reflexivity. At successor n, the goal becomes equality of successors of $n + 0$ and n; apply congruence of successor to the recursive proof for n. The recursive call is structurally smaller.

Check your argument. Explain both the conversion failure and termination of the repaired recursion.

MM8036-U8.09 · C · 30 min
Learning objective: U8.2 Transport in code

Describe how to cast a vector of length m to one of length n given $p : \mathrm{Id}_{\mathrm{Nat}(m,n)}$. State its reflexive computation.

Decisive idea. Transport in $k \mapsto \mathrm{Vec}(A, k)$.

Define cast by $\mathrm{transport}_{\lambda k.\mathrm{Vec}(A,k)}(p, v)$. Its result type is $\mathrm{Vec}(A, n)$. If p is reflexivity, cast computes to the input vector. No elements are inserted or removed: the equality proof justifies the change of index.

Check your argument. Distinguish reindexing along equality from changing the vector's data.

MM8036-U8.10 · A · 50 min
Learning objective: U8.1 CCC and dependent models

Explain why a cartesian closed category by itself supplies a model of simple arrows but does not automatically provide all the dependent-type structure used here.

Decisive idea. Dependent types vary over contexts and must reindex.

A CCC supplies products, terminal object, and exponentials, interpreting simple contexts and function types. A dependent model additionally needs types over each context, substitution/reindexing of those types and terms, context comprehension, and stable interpretations of the selected Pi, Sigma, identity, natural-number, and universe rules. CCC structure alone does not specify these indexed families or their stability laws. The lecture's families-of-sets construction supplies the extra data explicitly for its universe-free fragment.

Check your argument. List needed structures without claiming every CCC automatically has them.

MM8036-U8.11 · A · 50 min
Learning objective: U8.2 Constructive proof audit

Construct a program of type $(A + (A \to 0)) \to (((A \to 0) \to 0) \to A)$. Explain both branches, why this does not establish unrestricted double-negation elimination, and why replacing the missing decision input by a postulate would violate the completed-lab contract.

Decisive idea. In the negative branch, the double negation produces an impossible input to empty-type elimination.

Given decision $d : A + (A \to 0)$ and $h : (A \to 0) \to 0$, inspect d. In the left branch it contains $a : A$, so return a. In the right branch it contains $n : A \to 0$; application gives $h(n) : 0$, and empty-type elimination supplies the required result of type A. Both branches therefore have the same target, yielding a total case-defined function. The construction requires a decision input; it does not construct $A + (A \to 0)$ uniformly for arbitrary A, so it proves only conditional double-negation elimination. Assuming the missing decision by a postulate would assert new evidence rather than build it, and the pinned safe checker rejects that shortcut.

Check your argument. Type both branches, identify the additional input, and distinguish a constructed proof from an assumed axiom.

MM8036-U8.12 · A · 50 min
Learning objective: U8.1 Synthesis of substitution and semantics

In the families-of-sets model, prove that reindexing a term along $\sigma : \Delta \to \Gamma$ and then $\tau : \Theta \to \Delta$ equals reindexing along $\sigma \circ \tau$. Relate this to the syntactic explicit-substitution equation.

Decisive idea. Evaluate both constructions at an arbitrary θ.

First reindexing gives $t'(\delta) = t(\sigma(\delta))$ in $A(\sigma(\delta))$. Reindexing again gives $t''(\theta) = t(\sigma(\tau(\theta)))$ in $A(\sigma(\tau(\theta)))$. Direct reindexing along the composite gives precisely the same value in precisely the same fiber. Equality at each θ proves equality of the set sections. This realizes $t[\sigma][\tau] = t[\sigma \circ \tau]$, including its dependent typing, and shows why composition order is forced.

Check your argument. Check equality of fibers as well as values.

9 Mock assessment 1

Allow 180 minutes. Attempt all six problems and explain every inference. Use the rubric in the handbook for self-assessment; this is original practice, not an official examination.

MM8036-M1.01 · E · 15 min

Learning objective: U1–U2 Binding and reduction

Compute $\mathrm{FV}((\lambda x.\lambda y.xy)z)$ and reduce the term to normal form.

Decisive idea. The free z does not conflict with y.

The free-variable set is $\{z\}$. One beta step gives $\lambda y.zy$, which is beta normal because the application head is the variable z.

Check your argument. Track scope and establish normality.

MM8036-M1.02 · E · 15 min

Learning objective: U3–U4 Typing and logic

Derive the type $(A \to B) \to A \to B$ and interpret the term as a logical proof.

Decisive idea. Apply the function to the input.

The term $\lambda f.\lambda a.fa$ has the type: under $f : A \to B$, $a : A$, application gives B, and two abstractions discharge the assumptions. It is implication elimination followed by introduction.

Check your argument. Provide the intermediate judgment.

MM8036-M1.03 · C · 30 min

Learning objective: U2–U4 Normalization

Give a weakly but not strongly normalizing term, then explain why it cannot be typable in the arrow-only core.

Decisive idea. Combine the explicit infinite path with strong normalization.

$(\lambda x.\lambda y.y)\Omega$ reduces at the outside to a normal identity abstraction, but can reduce forever inside Ω. The strong-normalization theorem excludes an infinite full beta path from every term typable in the specified arrow-only core. Thus this source term is untypable, though its normal form is typable.

Check your argument. Use the theorem only for the fragment in which it was proved.

MM8036-M1.04 · C · 30 min
Learning objective: U5–U6 Dependent data

Define vector append on paper and justify its result index in both cases.

Decisive idea. Its length recursion follows addition on the first argument.

For lengths m, n, the output length is $\mathrm{add}(m, n)$. On nil return the second vector, whose length n is definitionally $\mathrm{add}(0, n)$. On cons, retain the head and append the tail recursively; the result length is $\mathrm{suc}(\mathrm{add}(m, n)) = \mathrm{add}(\mathrm{suc}(m), n)$ by definition. Each recursive call decreases the first vector.

Check your argument. State the signature, both clauses, and both conversions.

MM8036-M1.05 · A · 50 min
Learning objective: U3–U4 Metatheory synthesis

In the arrow-only language with sole constant $c : o$, show every closed term of type $o \to o$, applied to c, reduces to c. Explain what this does and does not say about equality of functions.

Decisive idea. Testing one closed argument is not a general extensionality rule.

Application gives $tc : o$ in the empty context. Normalization, preservation, and classification of closed base normal forms imply $tc \xrightarrow{*} c$. This establishes the stated observation at the only closed base normal datum. It does not assert judgmental equality of all functions: for example $\lambda x.x$ and $\lambda x.c$ are distinct beta-normal terms and hence not beta convertible, although both send c to c.

Check your argument. Give the canonicity argument and the explicit function counterexample.

MM8036-M1.06 · A · 50 min
Learning objective: U5–U8 Proof and model

Construct the dependent-pair map of U5.06 and explain how the same formula acts on total spaces in the set model. Prove it respects the projection to the new base.

Decisive idea. Project after mapping and compare with applying f after projection.

The term sends (a, b) to $(f(a), g(a)(b))$, whose second coordinate belongs to $B'(f(a))$. In sets this is a function between the two total spaces for exactly the same reason. Projecting its output gives $f(a)$, which equals f applied to the input's first projection. Thus the square of total-space map and base map f commutes pointwise.

Check your argument. Verify fiber typing and the commuting square, not just the output notation.

10 Mock assessment 2

Allow 180 minutes. Attempt all six problems and explain every inference. Use the rubric in the handbook for self-assessment; this is original practice, not an official examination.

MM8036-M2.01 · E · 15 min
Learning objective: U1–U3 Substitution
Reduce $(\lambda x.\lambda z.xz)y$ and give its type under $y : A \to B$.
Decisive idea. The resulting abstraction applies y.
The reduction gives $\lambda z.yz$. Under $z : A$, application yields $yz : B$, so abstraction gives $A \to B$ in the context $y : A \to B$.
Check your argument. Supply the context rather than treating the free y as closed.

MM8036-M2.02 · E · 15 min
Learning objective: U5–U6 Identity
Give the type and defining reflexive case of transport in a family $P : A \to U_0$.
Decisive idea. At reflexivity it is the identity function.
For $a, b : A$ and $p : \mathrm{Id}_{A(a,b)}$, transport has type $P(a) \to P(b)$. Its reflexive clause sends $u : P(a)$ to u. Path induction extends this clause to arbitrary p.
Check your argument. Include the direction and the role of path induction.

MM8036-M2.03 · C · 30 min
Learning objective: U4–U5 Constructive evidence
Construct a term taking $\Sigma_{x:A}(B(x) \times C(x))$ to $(\Sigma_{x:A}B(x)) \times (\Sigma_{x:A}C(x))$. Explain why the reverse direction is generally unavailable.
Decisive idea. Two independent existential witnesses need not coincide.
Send $(a, (b, c))$ to $((a, b), (a, c))$. In the reverse direction the two supplied witnesses may differ. For a set-model counterexample, let $A = \{0, 1\}$, let B be inhabited only at zero and C only at one. Both separate dependent sums are inhabited, while no index supports both fibers, so the original Sigma is empty. Soundness rules out a uniform reverse construction for all such families.
Check your argument. Give the term and a concrete model showing the missing common witness.

MM8036-M2.04 · C · 30 min
Learning objective: U6 Induction

Prove that mapping the identity function over a vector returns the original vector, using propositional equality.

Decisive idea. Use congruence of the fixed-head cons constructor.

At nil, map computes to nil and reflexivity proves the equality. At $\mathrm{cons}(a, v)$, map computes to $\mathrm{cons}(a, \mathrm{map}(\mathrm{id}, v))$. The induction hypothesis identifies its tail with v; congruence under $w \mapsto \mathrm{cons}(a, w)$ gives equality with $\mathrm{cons}(a, v)$. The result lives in the same length-indexed vector type on both sides.

Check your argument. Preserve the index and identify the congruence function.

MM8036-M2.05 · A · 50 min
Learning objective: U2–U4 Confluence and typing

Suppose a closed simply typed term reduces along two paths to u, v. Prove u, v have a common normal form of the original type.

Decisive idea. Normalize both reducts and then compare them.

Preservation gives both reducts the original type; closedness is retained by reduction. Strong normalization gives normal reducts n_u, n_v. Both are normal forms of the original source, so confluence forces them to be alpha equivalent. Repeated preservation gives that common normal form the original type. The argument uses confluence for agreement and normalization for existence; neither theorem substitutes for the other.

Check your argument. State the role of all three metatheorems.

MM8036-M2.06 · A · 50 min
Learning objective: U6–U8 Equality audit

Explain why the following inference is invalid: "The set model identifies all proofs of equality; therefore the paper theory may use K, and the labs may omit `--without-K`." Give a correct relationship among syntax, semantics, and the lab options.

Decisive idea. A model can validate additional principles that the syntax does not assume.

Semantic identification does not reflect back to a syntactic derivation without a suitable completeness theorem. Our intensional syntax uses J and has not assumed K. The set model validates the chosen rules but may forget distinctions between proofs. The labs deliberately check a corresponding restricted fragment with `--without-K`; omitting that flag would change the allowed pattern-matching discipline and would no longer certify the stated course constraint. Their existing proofs do pass with the restriction.

Check your argument. Identify the failed converse and distinguish changing a checker option from proving a new theorem.